Neural ODE based modeling of memristive devices

Karol Bednarz, Bartłomiej Garda, Zbigniew Galias, *Senior Member, IEEE*

***Abstract*—The development of modern memristive technologies requires accurate dynamic models to enable reliable simulation and prototyping in applications such as resistive RAM, neural networks, and neuromorphic computing. Existing symbolic models often exhibit limitations in accurately capturing the complex dynamic behavior of physical devices across various operating conditions, impacting their reliability in simulations. This work proposes a neural-network-based modeling of real memristors behaviour using Neural Ordinary Differential Equation (Neural ODE) models. Prior neural-network-based studies were limited by training data derived solely from theoretical models, preventing reliable assessment on real devices. The proposed method leverages a backpropagation technique based on solving the corresponding adjoint ODE backward in time using adjoint sensitivity methods. Experimental data obtained directly from Self-Directed Channel (SDC) memristors are used to train and validate the model. Results show that the Neural ODE approach more accurately reproduces the device's dynamic characteristics and provides improved flexibility and robustness compared to traditional theoretical models. These findings demonstrate the feasibility of Neural ODE–based frameworks for data-driven modeling of a broad class of physical memristive devices.**

***Index Terms*—Memristor modeling, artificial neural network, neural ODE.**

I. Introduction

NUMEROUS memristor models have been proposed following the discovery of memristive behavior at the nanoscale [1]. Physics-based approaches range from generalized device models capturing threshold-dependent switching [2], [3] to compact representations of RRAM structures such as the Stanford model [4]. For Self-Directed Channel (SDC) memristors specifically, the Generalized Metastable Switch (GMS) model [5] and its derivatives—the Mean Metastable Switch (MMS) [6] and Generalized MMS (GMMS) [7]—provide semi-empirical formulations based on transition probabilities between metastable states.

Despite their widespread adoption, these symbolic models share common limitations: they require extensive parameter tuning, employ window functions that introduce numerical discontinuities [8], and exhibit limited generalization when operating conditions deviate from calibration regimes [9]. A recent attempt to address these issues using Physics-Informed Neural Networks [10] demonstrated the potential of data-driven approaches, but relied solely on synthetic data generated from existing theoretical models rather than physical device measurements.

In [11], the authors introduced Neural Ordinary Differential Equations (Neural ODEs), where hidden-state dynamics are governed by an ODE with a neural network defining the vector field. Training is performed end-to-end using adjoint sensitivity methods for efficient gradient computation.

This paper proposes two Neural ODE architectures for modeling memristive devices trained directly on experimental data from SDC memristors. The first, **SingleNN-MemODE**, uses a physics-informed memductance with a neural network modeling only the state dynamics. The second, **DualNN-MemODE**, employs neural networks for both memductance and state dynamics, enabling a higher-dimensional latent representation. We demonstrate that both models outperform the MMS and GMMS benchmarks in prediction accuracy while maintaining comparable or shorter training times.

The paper is organized as follows. Sec. II presents the proposed architectures, the objective function, and the reference models. Sec. III reports experimental results and comparative analysis. The last section concludes the study.

The remaining part of this paper is organized as follows. In Sec. II, the neural network architecture is presented and two neural network-based models (SingleNN-MemODE and DualNN-MemODE) are proposed. The objective function defining the optimization problem to be solved is defined and existing memristors models used for comparison are briefly described. In Sec. III, proposed models are trained using experimental data to obtain accurate models of real SDC memristors. The results are compared with the results obtained using existing methods. The influence of hyperparameters of the proposed methods on their performance is discussed. The last section concludes the study.

II. Modeling Memristors using Neural Networks

This study utilizes experimental data obtained from Self-Directed Channel (SDC) memristors doped with tungsten. The structure and properties of these devices have been described in detail in the literature, e.g., in [12].

The experimental setup, illustrated in Fig. 1, consists of an SDC memristor connected in series with a resistor. The sinusoidal input voltage is supplied by an arbitrary waveform generator. Voltage signals are measured using a data acquisition (DAQ) system. Measurements are carried out for the following combinations of supply voltage amplitudes and frequencies: $V_s \in \{0.5,\ 1.0,\ 1.5\}$ [V], $f \in \{1,\ 5,\ 10,\ 20,\ 50,\ 100\}$ [Hz].

The analyzed model of a memristor follows the general theoretical framework of memristive devices originally introduced in [13], and is expressed as

$$i(t) = \mathcal{G}\left(\mathbf{x}(t), v(t)\right) v(t), \tag{1a}$$

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = f\left(\mathbf{x}(t), v(t)\right), \tag{1b}$$

where $\mathbf{x}$ denotes the vector of internal state variables, whose temporal evolution is governed by the function f, and $\mathcal{G}$

Manuscript received Month ??, 2026. This work was supported by the National Science Centre, Poland under Grant 2024/53/B/ST7/03841.

The authors are with the Department of Electrical Engineering, AGH University of Kraków, al. Mickiewicza 30, 30–059, Kraków, Poland, (e-mail: kbednarz@agh.edu.pl).

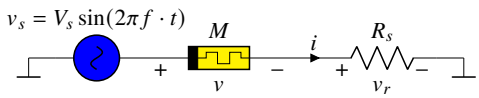

Fig. 1. Schematic diagram of the measurement setup used for the SDC memristor. The device is connected in series with a resistor, and the input voltage is applied via an arbitrary waveform generator. Voltage measurements are acquired using a data acquisition (DAQ) system.

represents the memductance of the device. The fundamental challenge in memristor modeling lies in accurately identifying the functional forms of $\mathcal{G}(\mathbf{x}, v)$ and $f(\mathbf{x}, v)$.

A. Objective Function

The objective function $\mathcal{L}$ is defined as the sum of four components:

$$\mathcal{L} = \lambda_{\text{base}}\mathcal{L}_{\text{b}} + \lambda_{\text{vel}}\mathcal{L}_{\text{v}} + \lambda_{\text{curv}}\mathcal{L}_{\text{c}} + \sum_{i=1}^{N_c} \lambda_{\text{con}_i}\mathcal{L}_{\text{con}_i}(x). \quad (2)$$

The terms $\mathcal{L}_{\text{b}}$, $\mathcal{L}_{\text{v}}$, and $\mathcal{L}_{\text{c}}$ represent the MSE of signal values, first derivatives, and second derivatives, respectively, between predicted and measured trajectories. The term $\mathcal{L}_{\text{con}}(x)$ enforces physical constraints. The weighting coefficients are computed at each step using the Geometric Loss Strategy (GLS) [14].

Let $z_k = (z_{k,n})_{n=1}^{N}$ and $\tilde{z}_k = (\tilde{z}_{k,n})_{n=1}^{N}$ denote measured and predicted sequences for variable $z \in \{v, i, \dot{v}, \dot{i}, \ddot{v}, \ddot{i}\}$, where $k = 1, \ldots, N_{\text{traj}}$ is the trajectory index and N is the number of samples. The normalized mean squared error is

$$\text{NMSE}(z_k, \tilde{z}_k) = \frac{1}{N}\sum_{n=1}^{N}\left(\frac{z_{k,n}}{\sigma_{z,k}} - \frac{\tilde{z}_{k,n}}{\sigma_{z,k}}\right)^2, \quad (3)$$

where normalization by the per-trajectory standard deviation $\sigma_{z,k}$ prevents amplitude disparities from biasing the optimization. The individual loss components are

$$\mathcal{L}_{\text{b}} = \frac{1}{N_{\text{traj}}}\sum_{k=1}^{N_{\text{traj}}}\left(\text{NMSE}(v_k, \tilde{v}_k) + \text{NMSE}(i_k, \tilde{i}_k)\right), \quad (4)$$

$$\mathcal{L}_{\text{v}} = \frac{1}{N_{\text{traj}}}\sum_{k=1}^{N_{\text{traj}}}\left(\text{NMSE}(\dot{v}_k, \tilde{\dot{v}}_k) + \text{NMSE}(\dot{i}_k, \tilde{\dot{i}}_k)\right), \quad (5)$$

$$\mathcal{L}_{\text{c}} = \frac{1}{N_{\text{traj}}}\sum_{k=1}^{N_{\text{traj}}}\left(\text{NMSE}(\ddot{v}_k, \tilde{\ddot{v}}_k) + \text{NMSE}(\ddot{i}_k, \tilde{\ddot{i}}_k)\right). \quad (6)$$

The constraint loss for the admissibility interval $x \in [a, b]$ uses smooth sigmoid transitions to maintain differentiability:

$$\mathcal{L}_{\text{con}}(x) = \mathbb{E}\left[\sigma\left(\frac{a-x}{\gamma}\right)(a-x)^2 + \sigma\left(\frac{x-b}{\gamma}\right)(x-b)^2\right], \quad (7)$$

where $\sigma(z) = (1+e^{-z})^{-1}$, γ controls transition steepness, and $\mathbb{E}[\cdot]$ denotes the ensemble average.

B. Neural ODE based Memristor Modeling

In this study, two neural network-based modeling approaches are proposed called. The first one, denoted as **SingleNN-MemODE**, employs a deterministic formulation of the memristor memductance $\mathcal{G}$, while the function f defining the dynamics of internal variables is modeled using a neural network. In the second one, referred to as **DualNN-MemODE**, both $\mathcal{G}$ and f are modeled using neural networks. The conceptual architecture of the proposed approaches is illustrated in Fig. 2.

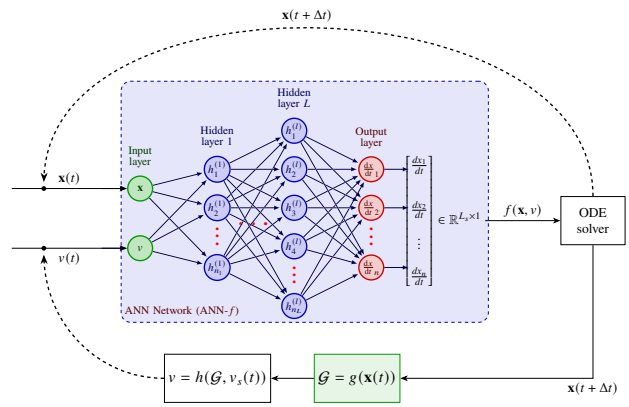

Fig. 2. The MemODE approach. The artificial neural network (ANN) evaluates the function $f(x, v)$, which is then integrated by the ODE solver to predict the state at the next time step $x(t + \Delta t)$. The upper dashed feedback loop indicates the recurrent nature of the process, where the output state is fed back as input for subsequent predictions. The memductance $\mathcal{G}$ is defined as function g which can be a deterministic function based on the internal variable x (as in SingleNN-MemODE) or the neural network with input $\mathbf{x}(t)$ (as in DualNN-MemODE). The function $h(\mathcal{G}, v_s(t))$ computes the memristor voltage v given the memductance $\mathcal{G}$ and supply voltage $v_s(t)$.

For the SingleNN-MemODE approach, the function f is implemented using an artificial neural network with a single output. For the DualNN-MemODE approach the number of outputs is equal to the number of internal variables and may be larger than 1.

In the SingleNN-MemODE approach, the memductance is defined on the basis of the physical mechanisms underlying self-directed channel (SDC) devices, which is expressed as a function of the internal state variable x as follows:

$$\mathcal{G}(x) = \frac{x}{R_{\text{ON}}} + \frac{1-x}{R_{\text{OFF}}}, \quad (8)$$

where R_{ON} and R_{OFF} denote the resistances in the low-resistance and high-resistance states, respectively. During the optimization process, the network parameters (weights $w_{k,i}$ and biases b_k) are tuned jointly with the values of R_{ON} and R_{OFF}. For this approach the following constraints are applied:

- $R_{\text{ON}} \in (500\,\Omega, R_{\text{OFF}})$,
- $R_{\text{OFF}} \in (R_{\text{ON}}, 200\,\text{k}\Omega)$,
- $x(t) \in (0, 1)$.

The lower bound for R_{ON} and the upper bound for R_{OFF} are based on empirical measurements. Remaining constraints are consequences of physically meaningful values of parameters and variables.

In the DualNN-MemODEapproach, the memductance is modeled using the second neural network. This neural network

takes the current state $\mathbf{x}$ as the input and produces the corresponding memductance $\mathcal{G}(\mathbf{x})$ as the output. The memristor state $\mathbf{x}$ is not constrained to be one-dimensional, enabling a more flexible representation of the device's internal dynamics. The state variables are not restricted to any specific range, and hence no explicit constraints are applied to them. For this approach the only constraint is

- $\mathcal{G}(\mathbf{x}) \in (5\,\mu\mathrm{S}, 2\,\mathrm{mS})$.

These bounds are based on measurements.

The learning process involves iteratively adjusting the network parameters to minimize the loss function that quantifies the difference between predicted and target outputs. This is achieved through backpropagation algorithm combined with the gradient descent optimization, where gradients of the loss function with respect to each parameter are computed and used to update the weights and biases in the direction that reduces the overall error.

C. Neural Network Architecture

The neural network training is implemented using the `JAX` library [15] for array-oriented numerical computation and `Equinox` [16] for neural network construction in `Python`. The parameter gradients of the ODE solution are computed using adjoint sensitivity methods [11]. The `tsit5` adaptive Runge-Kutta solver is employed for its computational efficiency and numerical accuracy, with adaptive step-size control ensuring stable integration during training.

The neural network architecture, illustrated in Fig. 3, employs an expansion–compression (diamond-shaped) topology that incorporates a double reduction in dimensionality in both the initial and final hidden layers. This architectural configuration substantially decreases the overall number of trainable parameters and helps to mitigate vanishing and exploding gradient issues commonly encountered in deep architectures processing temporal sequences.

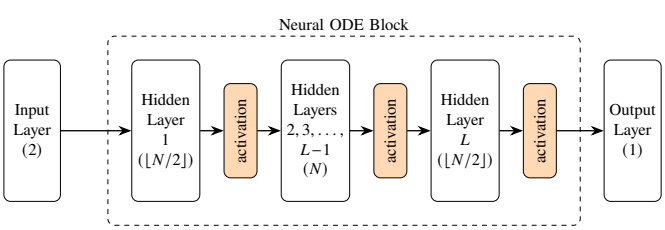

Fig. 3. Architecture of the feedforward neural network used within the Neural ODE framework for memristor dynamics modeling. The network consists of an input layer receiving two inputs, multiple hidden layers with decreasing-increasing-decreasing neuron counts $(\lfloor N/2 \rfloor, N, \ldots, N, \lfloor N/2 \rfloor)$, activation functions between layers, and a single output. The Neural ODE block encompasses the core computational layers that approximate the function $f(x(t), v(t))$ of the ODE system.

Several techniques are tested for the optimization of network weights.The choice of an optimizer and its parameters is guided by hyperparameter tuning conducted with the `Optuna` framework [17]. The hyperparameter search space comprises the learning rate, weight decay, batch size, and architecture parameters like the width and the depth of the neural network. The optimal configuration is selected based on minimizing the objective function on the validation set, ensuring that the model generalizes well to unseen data.

D. Existing Memristor Models for Comparison

The proposed models are compared with two established memristor models. The Mean Metastable Switch (MMS) model [6] describes the state dynamics as

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \frac{1}{\tau}\left(f_{\mathrm{ON}}(v)(1-x) - f_{\mathrm{OFF}}(v)x\right), \tag{9}$$

$$f_{\mathrm{ON}}(v) = \sigma\left(-\beta(v - V_{\mathrm{ON}})\right), \tag{10}$$

$$f_{\mathrm{OFF}}(v) = 1 - \sigma\left(\beta(v - V_{\mathrm{OFF}})\right), \tag{11}$$

where $\sigma(z) = (1 + e^{-z})^{-1}$, $\beta = q/(kT)$, and V_{ON}, V_{OFF} are the switching voltages. The memductance is defined by (8).

The Generalized MMS (GMMS) model [5], [7] extends MMS by modeling the memristor as a parallel connection of a memory-dependent element and a Schottky diode:

$$\begin{aligned} i(t) &= \eta\,\mathcal{G}(x)v + (1-\eta)\,i_{\mathrm{d}}(v), \\ i_{\mathrm{d}}(v) &= \alpha_{\mathrm{f}} e^{\beta_{\mathrm{f}} v} - \alpha_{\mathrm{r}} e^{-\beta_{\mathrm{r}} v}, \end{aligned} \tag{12}$$

where $\eta \in [0, 1]$ controls the ratio between memory-dependent and diode currents, and the state dynamics follow (9). The Schottky nonlinearity requires solving the DAE system $v_{\mathrm{s}} = R_{\mathrm{s}} i + v$ jointly with the state equation.

Parameter estimation for both models employs global search using Adaptive Differential Evolution followed by local L-BFGS-B refinement [18].

III. Results

The dataset consists of 18 trajectories obtained from measurements for all combinations of supply voltage amplitudes $V_{\mathrm{s}} \in \{0.5\,\mathrm{V}, 1.0\,\mathrm{V}, 1.5\,\mathrm{V}\}$ and frequencies $f \in \{1\,\mathrm{Hz}, 5\,\mathrm{Hz}, 10\,\mathrm{Hz}, 20\,\mathrm{Hz}, 50\,\mathrm{Hz}, 100\,\mathrm{Hz}\}$, with 10^5 samples per trajectory (100 periods, 1000 samples per period). The dataset is divided into 14 training and 4 validation trajectories: $(1.5\,\mathrm{V}, 5\,\mathrm{Hz})$, $(1\,\mathrm{V}, 1\,\mathrm{Hz})$, $(1\,\mathrm{V}, 100\,\mathrm{Hz})$, and $(0.5\,\mathrm{V}, 5\,\mathrm{Hz})$, selected to cover diverse operating conditions.

The optimal SingleNN-MemODE configuration, achieved through hyperparameters optimization, comprises four hidden layers (64, 128, 128, 64 neurons) trained with `AdamW` (learning rate 1.89×10^{-2}, batch size 2) using the swish activation function.

For DualNN-MemODE, the network computing $f(\mathbf{x}, v)$ has four hidden layers (120, 240, 240, 120 neurons) with 6 latent states, and the auxiliary network computing $\mathcal{G}$ has one hidden layer (80 neurons). Both are trained with `AdaBelief` (learning rate 6.30×10^{-3}, batch size 14), using GELU and ReLU activations, respectively.

A representative learning curve for SingleNN-MemODE is shown in Fig. 4. The consistent decrease of $\mathcal{L}$ on both training and validation sets over 360 epochs, along with their close agreement, confirms effective optimization and generalization.

The effectiveness of the full loss function $\mathcal{L}$ was verified against using only $\mathcal{L}_{\mathrm{b}}$: for SingleNN-MemODE, the validation loss decreased from $\mathcal{L}_{\mathrm{b}} = 2.24 \times 10^{-2}$ to 1.85×10^{-2} when including $\mathcal{L}_{\mathrm{v}}$ and $\mathcal{L}_{\mathrm{c}}$.

Representative test results are shown in Fig. 5 for sinusoidal excitation at 1.0 V, 1.0 Hz.

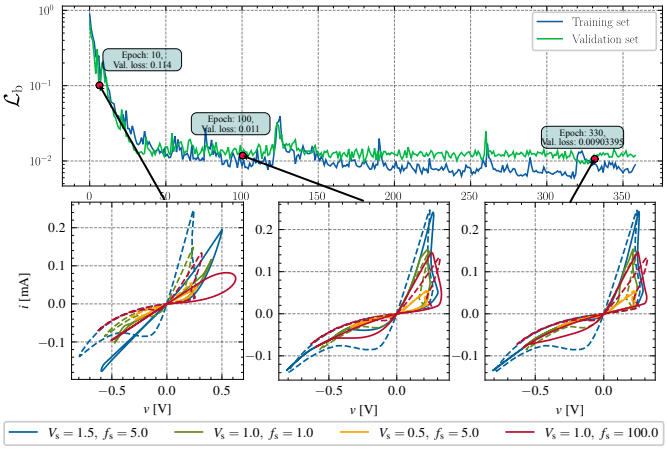

Fig. 4. Learning curve for the SingleNN-MemODE architecture over 360 epochs. Insets show v–i hysteresis loops at selected epochs (dashed: measured, solid: predicted).

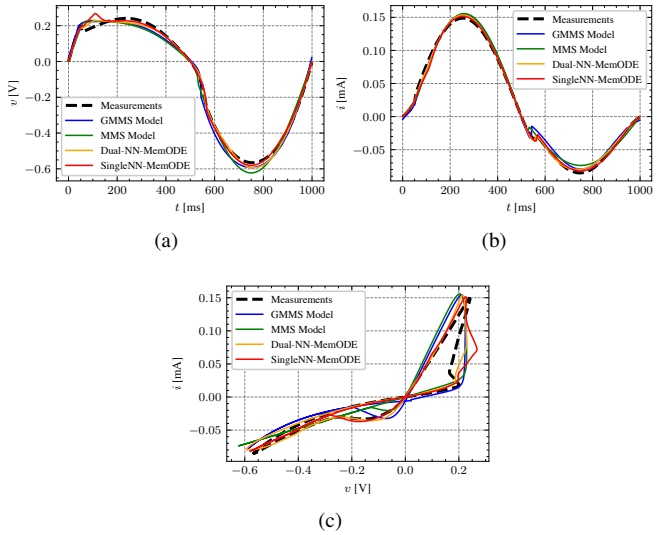

Fig. 5. Simulation test results for tungsten-doped memristor dynamics under sinusoidal voltage excitation (amplitude: 1.0 V, frequency: 1.0 Hz); (a) memristor's voltage waveform, (b) memristor's current response, (c) voltage-current hysteresis loop ($v - i$) illustrating the characteristic pinched behavior and memory properties fundamental to memristive operation.

A. Comparative Analysis with the MMS and GMMS Models

The proposed models are compared against the MMS [6] and GMMS [7] models, which serve as established benchmarks for SDC memristor simulation [19]. Fig. **??** presents a comparative analysis of the loss functions $\mathcal{L}_b$ obtained for the proposed and existing models, offering a quantitative assessment of their performance. The results highlight the neural network's enhanced capability to reproduce complex and subtle aspects of memristor dynamics that are insufficiently captured by the phenomenological MMS/GMMS framework.

Table II summarizes the average $\mathcal{L}_b$ values. The DualNN-MemODE model achieves the best validation loss of 5.77×10^{-3}, substantially outperforming both MMS and GMMS across all datasets.

Computation times are compared using a 3.5 GHz Intel i9-9900X processor. Training duration is 11.3 and 13.8 minutes for SingleNN-MemODE and DualNN-MemODE, respectively, compared to 20.4 minutes for MMS and 8.4 minutes for GMMS (using a two-stage global search followed by L-BFGS-B refinement [18]). However, trajectory generation (18 trajectories) takes approximately 0.05 s for MMS/GMMS versus 4–6 s for the neural models (with relative tolerance 10^{-3}, and absolute tolerance 10^{-6}).

TABLE I

COMPARATIVE ANALYSIS OF THE LOSS FUNCTION $\mathcal{L}_b$. VALUES ARE REPORTED FOR DIFFERENT SUPPLY VOLTAGE AMPLITUDES V_s AND FREQUENCIES f_s. ROW-WISE MINIMUM AND MAXIMUM LOSS VALUES ARE COLOR-CODED IN GREEN AND RED, RESPECTIVELY. RECORDS BELONGING TO THE VALIDATION DATASET ARE MARKED BY BOUNDING BOXES.

Loss function values for V_s = 0.5 V				
f_s [Hz]	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
1	9.31×10^{-3}	1.22×10^{-3}	3.94×10^{-2}	2.43×10^{-2}
5	3.40×10^{-3}	6.43×10^{-3}	2.97×10^{-2}	1.46×10^{-2}
10	3.05×10^{-3}	2.13×10^{-3}	2.85×10^{-2}	2.60×10^{-2}
20	3.79×10^{-3}	1.02×10^{-3}	3.25×10^{-2}	1.14×10^{-1}
50	2.03×10^{-2}	5.60×10^{-3}	3.68×10^{-2}	3.45×10^{-2}
100	2.82×10^{-2}	1.07×10^{-2}	9.91×10^{-2}	1.11×10^{-1}
Loss function values for V_s = 1 V				
f_s [Hz]	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
1	5.28×10^{-3}	4.47×10^{-3}	1.30×10^{-2}	1.63×10^{-2}
5	8.25×10^{-3}	2.67×10^{-3}	1.46×10^{-2}	1.71×10^{-2}
10	4.72×10^{-3}	8.86×10^{-4}	1.49×10^{-2}	1.58×10^{-2}
20	7.77×10^{-3}	5.53×10^{-3}	1.88×10^{-2}	1.90×10^{-2}
50	3.29×10^{-2}	7.37×10^{-3}	6.29×10^{-2}	6.22×10^{-2}
100	4.48×10^{-2}	8.58×10^{-3}	4.53×10^{-2}	3.25×10^{-2}
Loss function values for V_s = 1.5 V				
f_s [Hz]	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
1	1.74×10^{-2}	2.48×10^{-3}	1.75×10^{-2}	4.74×10^{-2}
5	1.57×10^{-2}	3.60×10^{-3}	2.15×10^{-2}	3.71×10^{-2}
10	1.55×10^{-2}	3.70×10^{-3}	2.08×10^{-2}	2.31×10^{-2}
20	1.32×10^{-2}	7.93×10^{-3}	2.07×10^{-2}	2.41×10^{-2}
50	1.37×10^{-2}	8.41×10^{-3}	1.69×10^{-2}	4.70×10^{-2}
100	2.64×10^{-2}	1.20×10^{-2}	5.54×10^{-2}	2.41×10^{-2}

TABLE II

COMPARATIVE ANALYSIS OF THE LOSS FUNCTION $\mathcal{L}_b$ FOR THE VALIDATION, TRAINING AND THE WHOLE DATASETS.

	SingleNN-MemODE	DualNN-MemODE	GMMS Model	MMS Model
Validation dataset				
$\mathcal{L}_b$	1.73×10^{-2}	5.77×10^{-3}	2.74×10^{-2}	2.51×10^{-2}
Training dataset				
$\mathcal{L}_b$	1.46×10^{-2}	5.11×10^{-3}	3.42×10^{-2}	4.22×10^{-2}
Whole dataset				
$\mathcal{L}_b$	1.52×10^{-2}	5.26×10^{-3}	3.27×10^{-2}	3.84×10^{-2}

IV. CONCLUSIONS

This work presented two novel neural ODE-based architectures for modeling the dynamics of memristor devices. The proposed

models leverage the power of neural networks to capture complex, nonlinear behaviors inherent in memristive systems, while maintaining the interpretability and physical consistency afforded by the ODE framework. The proposed models were applied for modeling of real Self-Directed Channel (SDC) memristors. Extensive evaluation against experimental data demonstrates that the neural ODE approach outperforms traditional phenomenological models, such as the MMS and GMMS models, in terms of accuracy and fidelity. The results highlight the capability of the neural ODE approach to learn intricate device dynamics directly from data.

Obtained results show that the computation time needed to optimize the proposed models is smaller than that for existing models. On the other hand the evaluation time needed to compute trajectories using the proposed models is significantly larger than for the MMS and GMMS models. The proposed models may also be used for modeling of memristors produced in other technologies. The findings of this study underscore the potential of neural ODEs as a versatile and effective framework for modeling complex physical systems, with applications in the design and analysis of memristor-based circuits. Developed methods pave the way to more accurate and efficient simulation tools, facilitating advancements in memristor technology and its integration into emerging computing paradigms, and open avenues for future research in data-driven modeling of nonlinear dynamical systems.

References

[1] D. B. Strukov, G. S. Snider, D. R. Stewart, and R. S. Williams, "The missing memristor found," *Nature*, vol. 453, no. 7191, pp. 80–83, May 2008. [Online]. Available: http://dx.doi.org/10.1038/nature06932

[2] C. Yakopcic, T. M. Taha, G. Subramanyam, R. E. Pino, and S. Rogers, "A memristor device model," *IEEE Electron Device Letters*, vol. 32, no. 10, pp. 1436–1438, Oct. 2011. [Online]. Available: http://dx.doi.org/10.1109/LED.2011.2163292

[3] S. Kvatinsky, M. Ramadan, E. G. Friedman, and A. Kolodny, "VTEAM: A general model for voltage-controlled memristors," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 62, no. 8, pp. 786–790, 2015.

[4] P.-Y. Chen and S. Yu, "Compact modeling of RRAM devices and its applications in 1T1R and 1S1R array design," *IEEE Transactions on Electron Devices*, vol. 62, no. 12, pp. 4022–4028, Dec. 2015. [Online]. Available: http://dx.doi.org/10.1109/ted.2015.2492421

[5] T. W. Molter and M. A. Nugent, "The generalized metastable switch memristor model," in *CNNA 2016; 15th International Workshop on Cellular Nanoscale Networks and their Applications*, 2016, pp. 1–2.

[6] L. Minati, L. Gambuzza, W. Thio, J. Sprott, and M. Frasca, "A chaotic circuit based on a physical memristor," *Chaos, Solitons & Fractals*, vol. 138, p. 109990, Sep. 2020. [Online]. Available: http://dx.doi.org/10.1016/j.chaos.2020.109990

[7] V. Ostrovskii, P. Fedoseev, Y. Bobrova, and D. Butusov, "Structural and parametric identification of Knowm memristors," *Nanomaterials*, vol. 12, no. 1, p. 63, Dec. 2021. [Online]. Available: http://dx.doi.org/10.3390/nano12010063

[8] Z. Galias, "Simulations of memristors using the Poincaré map approach," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 3, pp. 963–971, Mar. 2020. [Online]. Available: http://dx.doi.org/10.1109/tcsi.2019.2949736

[9] ——, "Study of amplitude control and dynamical behaviors of a memristive band pass filter circuit," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 65, no. 5, pp. 637–641, May 2018. [Online]. Available: http://dx.doi.org/10.1109/tcsii.2018.2820104

[10] Y. Lee, K. Kim, and J. Lee, "A compact memristor model based on physics-informed neural networks," *Micromachines*, vol. 15, no. 2, 2024. [Online]. Available: https://www.mdpi.com/2072-666X/15/2/253

[11] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *32nd Conference on Neural Information Processing Systems (NeurIPS 2018)*, Montréal, Canada, 2018.

[12] K. A. Campbell, "Self-directed channel memristor for high temperature operation," *Microelectronics Journal*, vol. 59, pp. 10–14, Jan. 2017. [Online]. Available: http://dx.doi.org/10.1016/j.mejo.2016.11.006

[13] L. Chua and S. M. Kang, "Memristive devices and systems," *Proceedings of the IEEE*, vol. 64, no. 2, pp. 209–223, 1976. [Online]. Available: http://dx.doi.org/10.1109/PROC.1976.10092

[14] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich, "GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, Jul. 2018, pp. 794–803. [Online]. Available: https://proceedings.mlr.press/v80/chen18a.html

[15] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[16] P. Kidger and C. Garcia, "Equinox: neural networks in JAX via callable PyTrees and filtered transformations," *Differentiable Programming workshop at Neural Information Processing Systems 2021*, 2021.

[17] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD'19. ACM, Jul. 2019, pp. 2623–2631. [Online]. Available: http://dx.doi.org/10.1145/3292500.3330701

[18] C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal, "Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization," *ACM Transactions on Mathematical Software*, vol. 23, no. 4, pp. 550–560, Dec. 1997. [Online]. Available: http://dx.doi.org/10.1145/279232.279236

[19] K. Bednarz and B. Garda, "Measurement and modeling of self-directed channel (SDC) memristors: An extensive study," *Energies*, vol. 17, no. 21, p. 5400, Oct. 2024. [Online]. Available: http://dx.doi.org/10.3390/en17215400